Distributed SQL Explained

Spanner, CockroachDB, Raft and TrueTime

Watch the 4-min video

In one line: distributed SQL (NewSQL) gives you SQL and serializable ACID transactions across many nodes and regions: data is split into ranges, each replicated with Raft, and clocks (TrueTime or hybrid logical clocks) order transactions; the price is higher write latency, so design for locality.

1 The problem: a global payments ledger

You are building a global payments ledger. Every transfer must be a real **ACID transaction**, with no double spending, ever. Customers are in Europe, America and Asia, and a **whole cloud region can fail** without losing a single committed transfer.

One Postgres server cannot do that: it lives in one region. A hand-sharded Postgres cannot do it easily either: transactions that span shards, rebalancing and region failover are all left to you.

2 What distributed SQL promises

NewSQL, or **distributed SQL**, offers the SQL and transactions of a relational database with the horizontal scale of NoSQL. The best-known examples are **Google Spanner**, **CockroachDB** and **YugabyteDB**. You talk SQL; the database handles **sharding, replication and rebalancing** for you.

3 Under the hood: ranges and Raft

Tables are split into **ranges**: chunks of sorted keys. Each range is copied to **three or more nodes**, and the copies are kept in sync with a **consensus protocol** such as **Raft**. One replica is the **leader**; a write commits once a **majority** of replicas have it. So any single node, or a whole zone, can fail without losing committed data. Ranges **split and move automatically** as data grows or nodes are added.

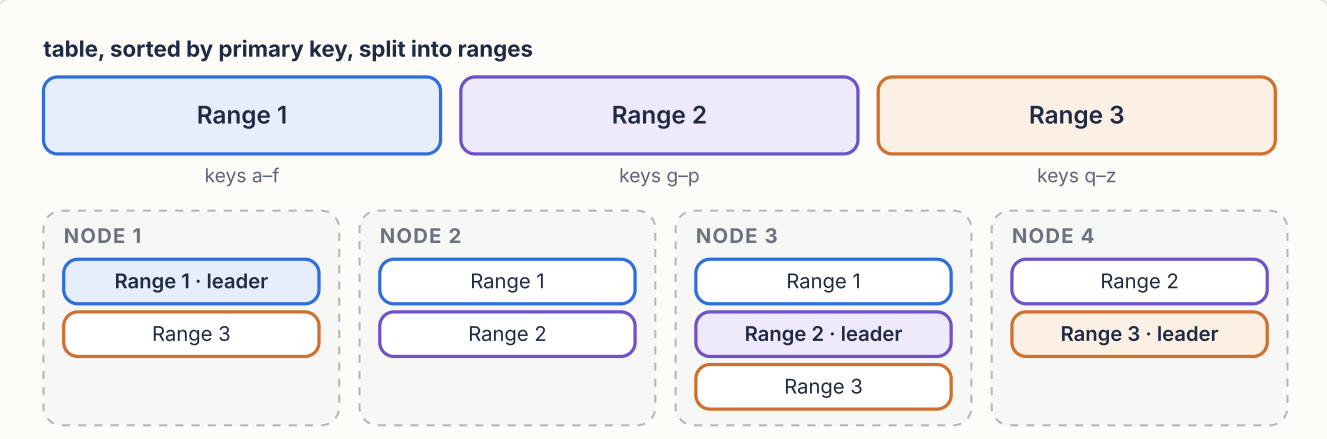


Figure 1. Three ranges, three replicas each, spread over four nodes. Each range has its own leader; losing any one node leaves every range with a majority.

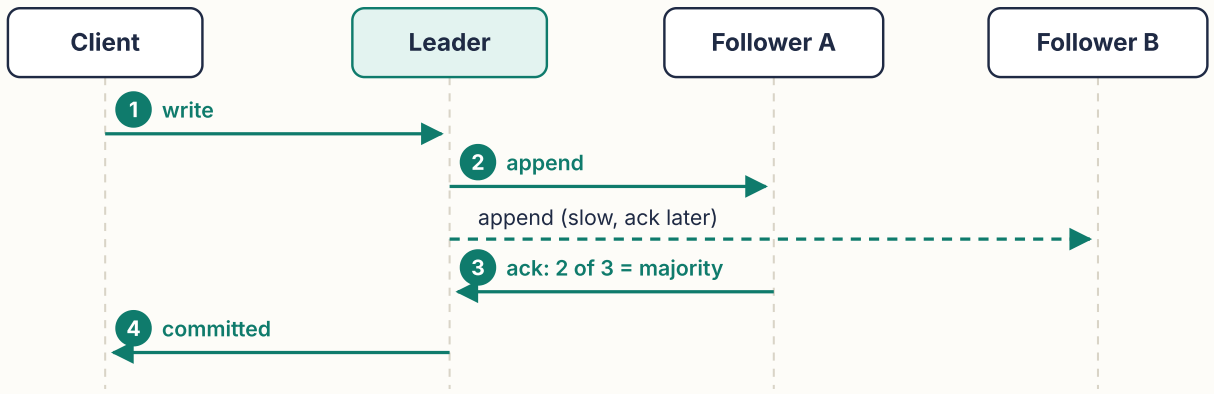


Figure 2. Raft: the leader commits as soon as 2 of 3 replicas (itself and A) have the write. Slow follower B catches up later.

Majority means **2 of 3** or **3 of 5**. With 3 replicas you survive 1 failure; with 5 you survive 2.

4 Ordering transactions across machines

A transaction can span many ranges on different machines, so the database needs a **consistent order of events across all of them**. Server clocks drift, so plain wall-clock time is not enough.

- **Spanner** uses **TrueTime**: GPS receivers and atomic clocks give every server the time with a **known uncertainty** (a few milliseconds). Spanner picks a commit timestamp and then **waits out that uncertainty** before making the commit visible (commit wait). Any transaction that starts afterwards is guaranteed a later timestamp.
- **CockroachDB** uses **hybrid logical clocks (HLC)**: physical time combined with a logical counter, plus a maximum allowed clock offset. When a read sees a value inside its uncertainty window, it restarts at a later timestamp instead of waiting on every commit.

Both give strong, **serializable** transactions.

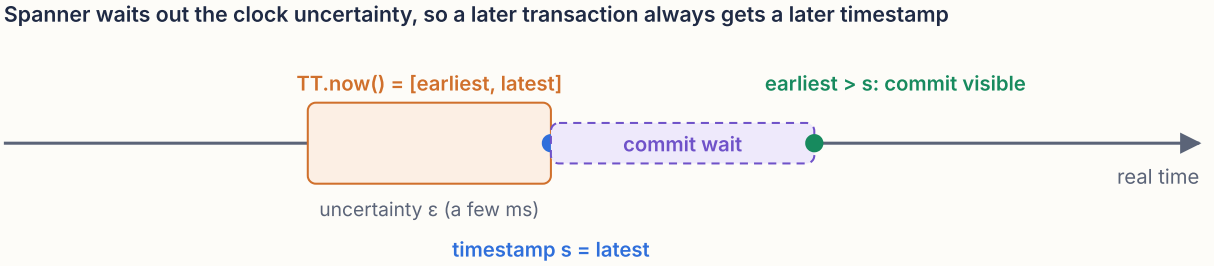


Figure 3. TrueTime commit wait: once even the earliest possible current time is past the timestamp, the commit is safely in the past everywhere.

5 The cost: latency, so design for locality

Every write needs a round trip to a majority of replicas. Within one region that is a **few milliseconds**. Across continents it can be **100 ms or more**. Transactions that touch several ranges need extra coordination (a commit protocol across leaders), which adds more round trips.

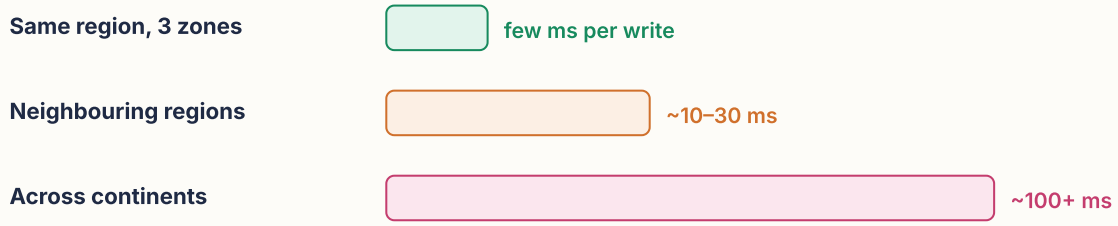


Figure 4. Where you place replicas decides your write latency. Illustrative numbers; measure your own.

So **locality matters**: keep each customer's data in their **home region**, so most transactions are local. Use **follower reads** for slightly stale but fast reads from the nearest replica, when a few seconds of staleness is fine.

WORKED EXAMPLE: THE LEDGER ON COCKROACHDB

The `accounts` table is **regional by row**: each row lives in its owner's home region, so a Paris customer's balance has its leader in Europe. The database has replicas in **three regions** and is set to survive a region failure.

```
ALTER DATABASE ledger SURVIVE REGION FAILURE;
ALTER TABLE accounts SET LOCALITY REGIONAL BY ROW;

BEGIN;   -- one serializable transaction, even across regions
UPDATE accounts SET balance = balance - 100 WHERE id = 'eu-123';
UPDATE accounts SET balance = balance + 100 WHERE id = 'us-456';
COMMIT;
```

A Paris-to-Paris transfer commits in a few ms. A Paris-to-New-York transfer touches two regions and may take ~100 ms or more. If a whole region goes down, the remaining two still form a majority, and **no committed transfer is lost**.

6 Distributed SQL or sharded Postgres?

Choose distributed SQL when...	Stay with Postgres when...
You need transactions across shards, with strong consistency	One primary (plus replicas) handles the load
You want automatic sharding and rebalancing	Simple sharding by one key is enough
You must survive zone or region failures with no data loss	The lowest possible write latency matters most
You don't want to build and run your own sharding layer	You depend on Postgres extensions and its ecosystem

COMMON MISTAKES

- Expecting single-node Postgres latency from a multi-region cluster.
- Ignoring locality, so every transaction crosses an ocean.
- Sequential primary keys, which send all inserts to one range (a hotspot); prefer UUIDs or hashed keys.
- Choosing it “for scale” when one Postgres primary would do.

7 Key terms

Distributed SQL / NewSQL: SQL + ACID, scaled out across nodes

Leader: replica that orders a range's writes

HLC: hybrid logical clock: physical + counter

Range: chunk of sorted keys, the unit of replication

TrueTime: Spanner's clock with bounded uncertainty

Regional by row: each row homed in its region

Raft: consensus: commit on a majority

Commit wait: wait out clock uncertainty before commit

Follower read: slightly stale read from a nearby replica

8 Interview questions

Q1. When would you choose distributed SQL over sharded Postgres? BASIC

When you need SQL and ACID transactions across many nodes or regions, strong consistency, high availability through consensus replication and automatic sharding, and would rather not build that yourself. Accept higher write latency and design for locality.

Q2. How does a distributed SQL database survive a node failure? BASIC

Each range is replicated, typically 3 ways, with Raft. A write commits on a majority (2 of 3), so losing one node leaves a majority with every committed write; a new leader is elected in seconds.

Q3. What is TrueTime? BASIC

Spanner's clock API, backed by GPS and atomic clocks, that returns time as an interval with a known uncertainty of a few ms. Spanner waits out that uncertainty before committing, so timestamps match real-time order.

Q4. Users in Asia see 150 ms writes on a cluster whose leaders are in the US. Fix? SCENARIO

Make data regional by row (or partition by region) so each user's rows have their leader in their region. Local writes then take a few ms; only cross-region transactions pay the long round trip.

Q5. How many replicas to survive a full region outage? SCENARIO

Replicas in at least 3 regions, so losing one still leaves a majority (2 of 3). With 5 replicas over 3 regions (2+2+1), losing any one region still leaves at least 3 of 5.

Q6. A product page needs fast reads worldwide and tolerates 5 s staleness. SCENARIO

Use follower reads: serve from the nearest replica at a slightly old timestamp, with no trip to the leader. Writes still go to the leader.

Q7. Why not just use NTP timestamps to order transactions? TRICKY

NTP clocks can be off by tens of ms or more, with no guaranteed bound, so a later transaction could get an earlier timestamp and break serializability. Spanner bounds the error (TrueTime); CockroachDB uses HLC with a maximum offset and uncertainty restarts.

Q8. Is distributed SQL always better than Postgres? TRICKY

No. Single-row writes are slower (consensus round trips), operations are more complex, and some Postgres features and extensions are missing. If one primary fits the load, Postgres is simpler and faster.

Next: SD-E-033 · How to Choose a Database backendroadmap.com